endava

endava

AI-Native Foundations: *From an AI Perspective*

AI in Test Analysis, Design and Creation of Test Cases.

Agenda

01	STLC – Software Testing Life Cycle	Pag. 4
02	Documents Review and Requirements Analysis	Pag. 5
03	AI-Driven Document Review and Requirement Analysis	Pag. 10
04	Test Analysis and Design Overview	Pag. 12
05	Test Techniques	Pag. 15
06	AI-Assisted Test Cases Creation	Pag. 22
07	Theoretical Quiz	Pag. 24
08	Practical case	Pag. 25

01 Basic Concepts: STLC – Software Testing Life Cycle

Test Planning → Analysis & Design → Test Implementation → Test Execution & Control → Test Completion

ISTQB® Certified Tester

Chapter

02 Documents Review and Requirements Analysis

Documents Review and Requirements Analysis from ISTQB:

Static Testing

- **Static Testing Definition:**

Static testing helps identify defects in the earliest phases, supporting the testing principle related to early testing. It enables the evaluation of the quality of work products such as documented requirements and system functional specifications by applying tests to non-executable work products.

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Documents Review and Requirements Analysis.

Work products in Static Testing

- Requirements specifications (BRD, SRS)
- Use cases
- User stories
- Technical designs
- Architecture diagrams
- Test plans
- Test cases

Typical Defects found in Static Testing

- Requirements defects
- Design defects
- Coding defects
- Deviations from standards
- Incorrect interface specifications
- Security vulnerabilities
- Test coverage issues

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Static Testing

Process Review.

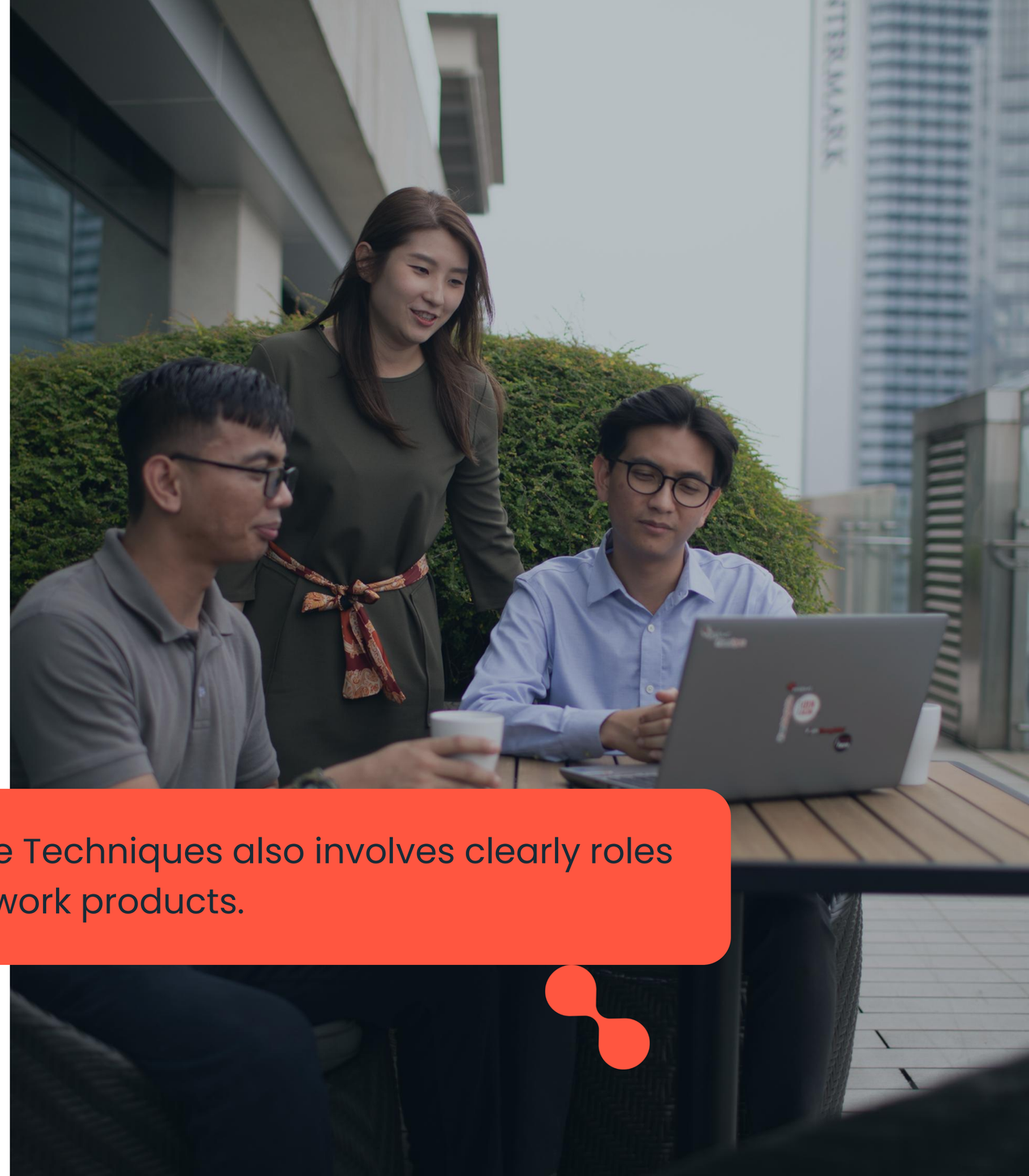
- Informal Review
- Formal Review
- Walkthroughs
- Technical Review
- Inspection

This process encompasses planning, execution, and reporting activities, as well as clearly defined roles and work products.

Static Testing

Review Techniques.

- Ad Hoc
- Checklists
- Scenario-Based
- Role-Based
- Perspective-Based



These Techniques also involves clearly roles and work products.

Chapter

03 AI-Driven Document Review and Requirement Analysis

AI-Driven Document Review and Requirement Analysis

How AI helps in document review

- identify ambiguous, incomplete, or contradictory requirements
- detect missing acceptance criteria
- find duplicated or inconsistent information across documents
- highlight unclear wording or non-testable statements
- suggest improvements in structure and clarity

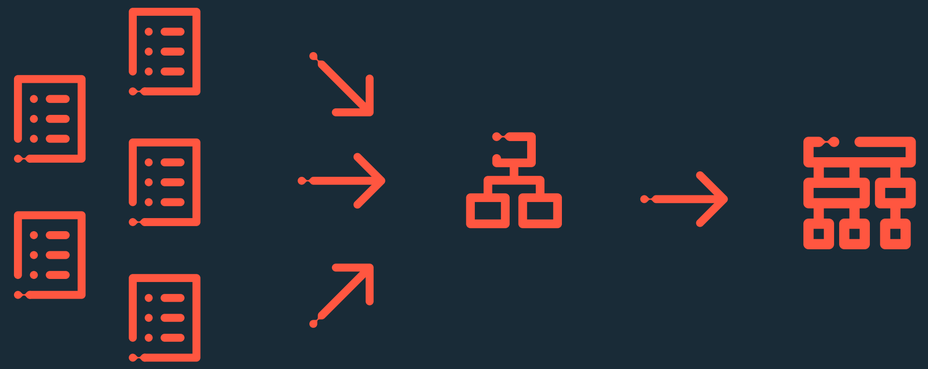
How AI helps in requirement analysis

- analyze the **test basis** and identify what should be tested
- derive possible **test conditions**
- suggest positive, negative, and edge-case scenarios
- detect requirement gaps and hidden assumptions
- support risk identification and prioritization

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

03

Test Analysis and Design Overview



The Test Analysis.

The Test Analysis means The information from the requirements and other work products is transformed into test conditions that will serve as the basis for designing test cases. At this stage, the tester answers the question: What should be tested?.

Activities:

- Analyzes the test basis, such as requirements, user stories, or designs.
- Identifies what needs to be tested.
- Defines test conditions.
- Evaluates risks associated with the requirements.
- Determines the necessary test coverage.

Eg: Requirement: “The system must allow bank transfers.”

Test Analysis Result: Possible situations: Transfer with sufficient balance. Transfer with insufficient balance. Transfer to an invalid account. Transfer outside permitted hours. Validation of daily transfer limits.

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

The Test Design.

The Test Design is the activity within the testing process in which the test conditions identified during Test Analysis are transformed into detailed test cases and test procedures.

Activities:

- Design test cases based on test conditions.
- Select and apply test design techniques
- Define the necessary test data.
- Design test procedures.
- Identify dependencies between tests.
- Determine coverage criteria.

Test Design Techniques: black-box, white-box, and experience-based.

Test Case Elements:

- Test Case ID
- Test Case Title
- Description
- Preconditions
- Test Data
- Test Steps
- Expected Result
- Actual Result
- Status

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

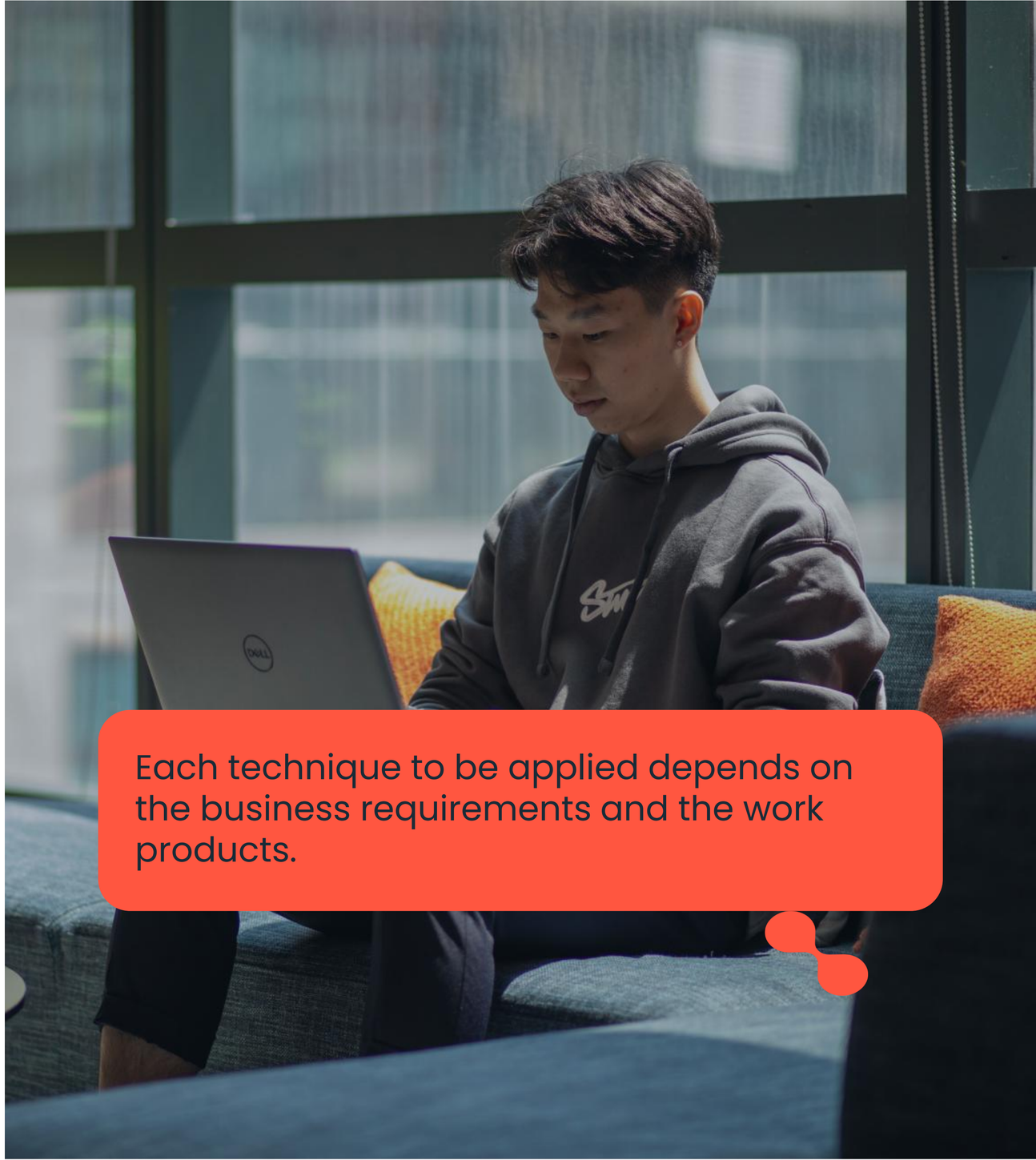
Chapter

04 Test Techniques

Test Techniques:

Black-Box Techniques

- **Equivalence Partitioning**
- **Boundary Value Analysis**
- **Decision Tables**
- **State Transition Testing**

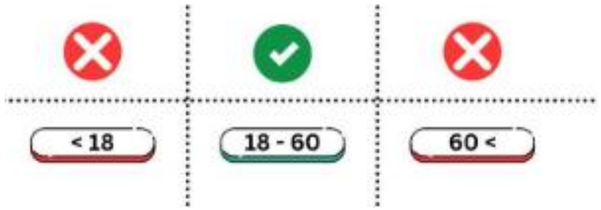


Each technique to be applied depends on the business requirements and the work products.

Test Techniques: Black-Box Techniques

Equivalence Partitioning

Divides data into partitions (known as equivalence partitions) based on the expectation that all elements within a given partition will be processed in the same way by the test object.



Boundary Value Analysis

To test the boundary values of equivalence partitions, as defects are more likely to occur at the boundaries of input ranges.

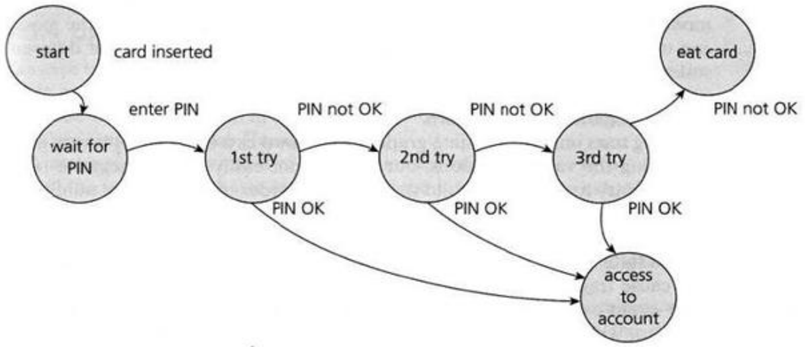
Decision Tables

Tests the implementation of system requirements that specify how different combinations of conditions result in different outcomes. Decision tables are an effective way to record complex logic, such as business rules.

Credit limit exceeded	Y	Y	Y	Y	N	N	N	N
Prompt payer	Y	Y	N	N	Y	Y	N	N
Special Clearance	Y	N	Y	N	Y	N	Y	N
Accept order	X	-	X	-	X	X	X	X
Reject order	-	X	-	X	-	-	-	-

State Transition Testing

Models the behavior of a system by showing its possible states and the valid state transitions.

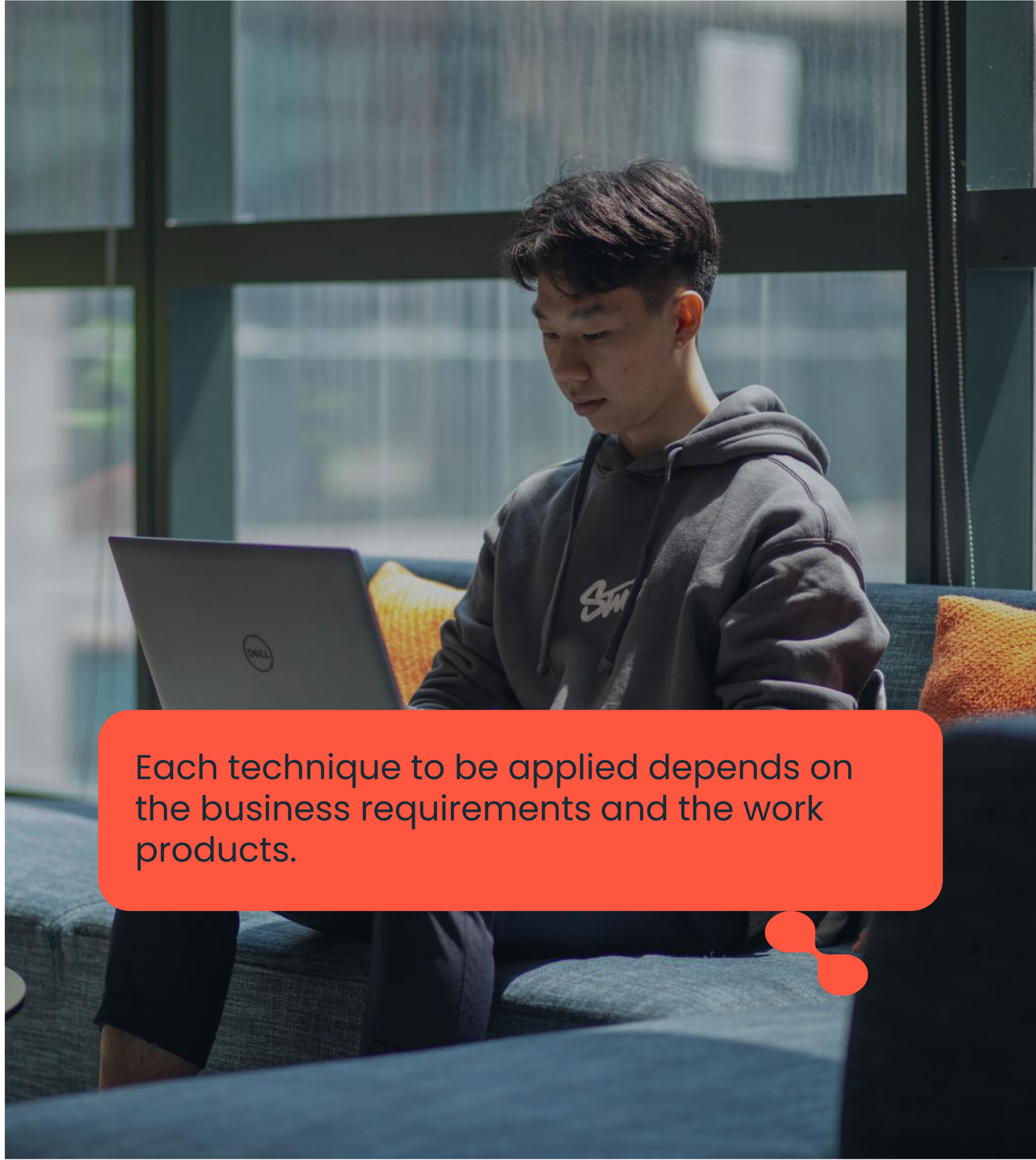


© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Test Techniques:

White-Box Techniques

- **Statement Coverage**
- **Branch Coverage**

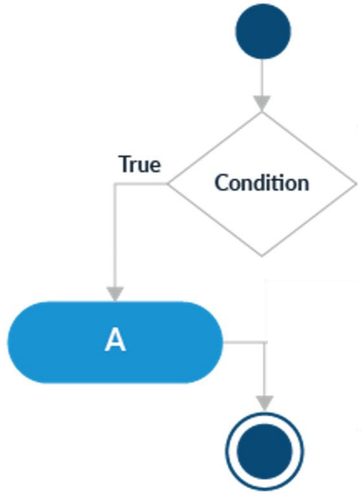


Each technique to be applied depends on the business requirements and the work products.

Test Techniques: White-Box Techniques

Statement Coverage

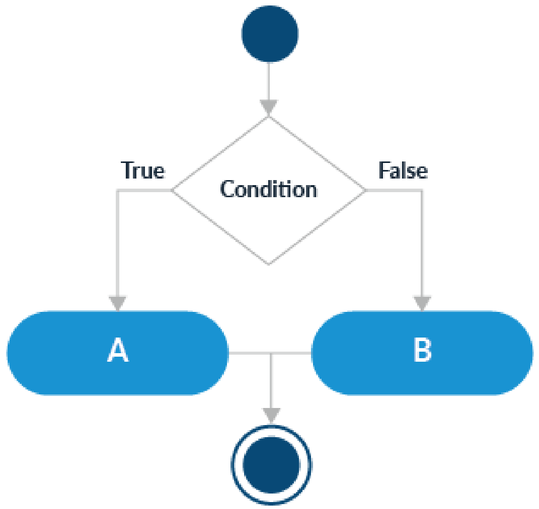
Tests coverage items are executable statements. The objective is to design test cases that exercise statements in the code until an acceptable level of coverage is achieved.




```python
def verificar_numero(num):
    if num > 0:
        print("Número positivo")
    print("Fin del programa")
```

Branch Coverage

Tests transfer of control between two nodes in the control flow graph, which shows the possible sequences in which the statements of the source code are executed in the test object. Each transfer of control may be unconditional (i.e., straight-line code) or conditional (i.e., a decision outcome).



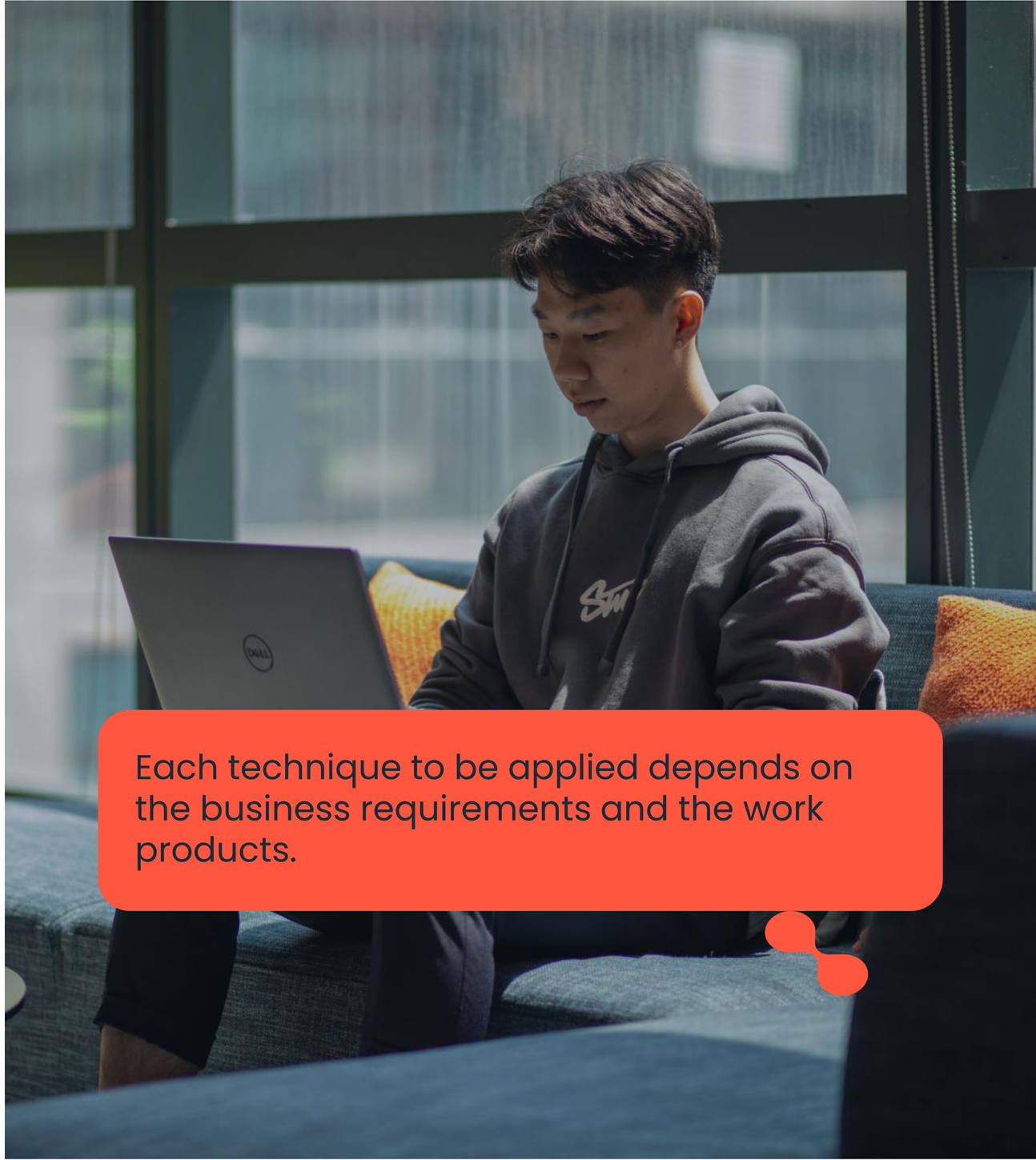

```python
def verificar_numero(num):
    if num > 0:
        print("Número positivo")
    else:
        print("Número negativo o cero")
```

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Test Techniques:

Experience-Based Techniques

- **Error Guessing**
- **Exploratory Testing**
- **Checklist Verification**



Each technique to be applied depends on the business requirements and the work products.

Know your audience

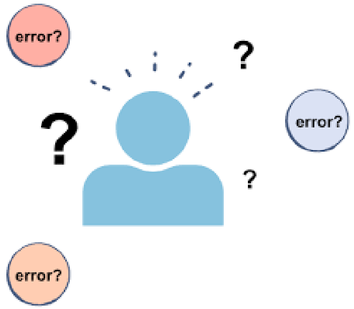
Test Techniques:

Experience-Based Techniques

Error Guessing

anticipates the occurrence of errors, defects, and failures, based on the tester's knowledge, including:

- How the application has functioned in the past.
- The types of errors developers tend to make
- The types of failures that have occurred in other similar applications.



Exploratory Testing

tests are designed, executed, and evaluated simultaneously while the tester learns about the test object. Testing is used to gain more knowledge about the test object, to explore it more deeply through focused testing, and to create tests for areas that have not yet been tested.

Checklist Verification

a tester designs, implements, and executes tests to cover the test conditions in a checklist. Checklists may be created based on experience, knowledge of what is important to the user, or an understanding of why and how the software fails.

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

05 AI-Assisted Test Cases Creation

AI-Assisted Test Cases Creation

How AI helps in Test Case Creation

- analyze requirements and identify test conditions
- generate positive, negative, and edge test cases
- suggest boundary values and equivalence partitions
- derive tests from business rules and acceptance criteria
- identify missing scenarios or gaps in coverage
- create test cases in structured formats such as steps, preconditions.
- help convert manual test cases into automation-ready formats

Benefits

- reduce manual effort
- speed up test design
- improve initial test coverage
- support consistency across test cases
- help testers focus on review, refinement, and risk-based prioritization

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Chapter

06 THEORETICAL QUIZ

Chapter

07 PRACTICAL CASE

Thank you!